

# Микросервисная система учета заказов в кафе

Данная презентация представляет собой обзор микросервисной системы для учета заказов в небольшом кафе. Проект направлен на автоматизацию и повышение эффективности работы персонала.



# Цель проекта и архитектура

## Основная цель

Автоматизация процесса приема и обработки заказов, обеспечение удобного веб-интерфейса для официантов и администраторов.

## Тип приложения

Single Page Application (SPA) без использования react-router для быстрой навигации без перезагрузки страницы.

## Архитектура

Микросервисная система, обеспечивающая масштабируемость и независимость компонентов.



# Возможности для всех пользователей

Система предоставляет базовые функции, доступные как авторизованным, так и неавторизованным пользователям, обеспечивая прозрачность и доступность информации о работе кафе.



## Просмотр меню

- Фильтрация по категориям
- Поиск по названию
- Отображение цен и доступности



## Просмотр заказов

- Список активных заказов
- Фильтрация по статусу



## Просмотр столиков

- Схема зала со статусами
- Информация о вместимости

# Функции для авторизованных пользователей

Для официантов и администраторов система предлагает расширенный набор функций, позволяющих эффективно управлять операционной деятельностью кафе.

## Официант



- Управление заказами
  - Создание новых заказов
  - Изменение статуса заказов
  - Добавление позиций в заказ
- Управление столиками
  - Изменение статуса столов (свободен/занят/забронирован)

## Администратор



- Все функции официанта

Дополнительно к базовым функциям официанта, администратор обладает полным контролем над настройками кафе.
- Управление меню (CRUD)
  - Добавление, редактирование, деактивация позиций
- Управление категориями (CRUD)
  - Создание, редактирование, сортировка
- Управление столиками (CRUD)
  - Добавление, редактирование, деактивация

# Технологический стек Frontend

Frontend-часть приложения реализована с использованием современных и эффективных технологий для обеспечения высокой производительности и удобства разработки.



React 19.2.0

Библиотека для создания пользовательского интерфейса.

2

TypeScript 5.9.3

Типизированный JavaScript для повышения надежности кода.



Vite

Современный инструмент сборки, обеспечивающий быструю разработку.



MobX 6.15.0

Библиотека для эффективного управления состоянием приложения.

5

SCSS/SASS

Препроцессор CSS для модульной и гибкой стилизации.



Firebase

База данных (Firebase Realtime Database) для хранения данных.



# Структура проекта: Frontend

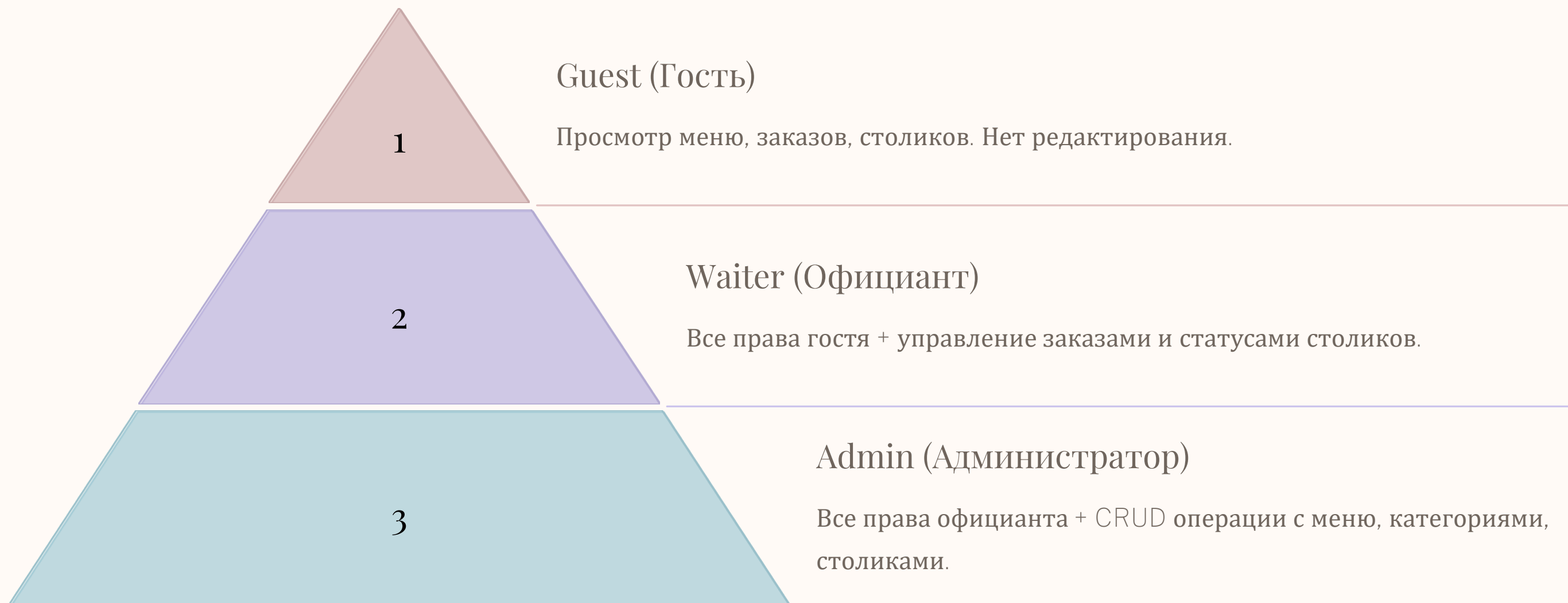
Проект имеет четко определенную и логичную структуру, что облегчает разработку, поддержку и масштабирование.

## Основные разделы

- components/: Переиспользуемые UI компоненты.
- pages/: Отдельные страницы приложения.
- store/: Управление состоянием (MobX stores).
- types/: TypeScript типы и интерфейсы.
- styles/: Глобальные стили приложения.
- firebase.ts: Конфигурация Firebase.
- App.tsx: Главный компонент приложения.

# Система ролей и прав доступа

Проект реализует гибкую систему ролей, обеспечивая соответствующий уровень доступа к функциям системы для каждого типа пользователя.



# Основные сущности данных

Данные в системе организованы в виде ключевых сущностей, что обеспечивает структурированное хранение и легкое управление информацией.

1

## MenuItem

id, name, description, categoryId,  
price, isAvailable, isActive.

2

## Category

id, name, description, sortOrder,  
isActive.

3

## Table

id, number, capacity, status,  
location.

4

## Order

id, tableId, tableNumber, items[], status, totalAmount,  
notes, createdBy.

5

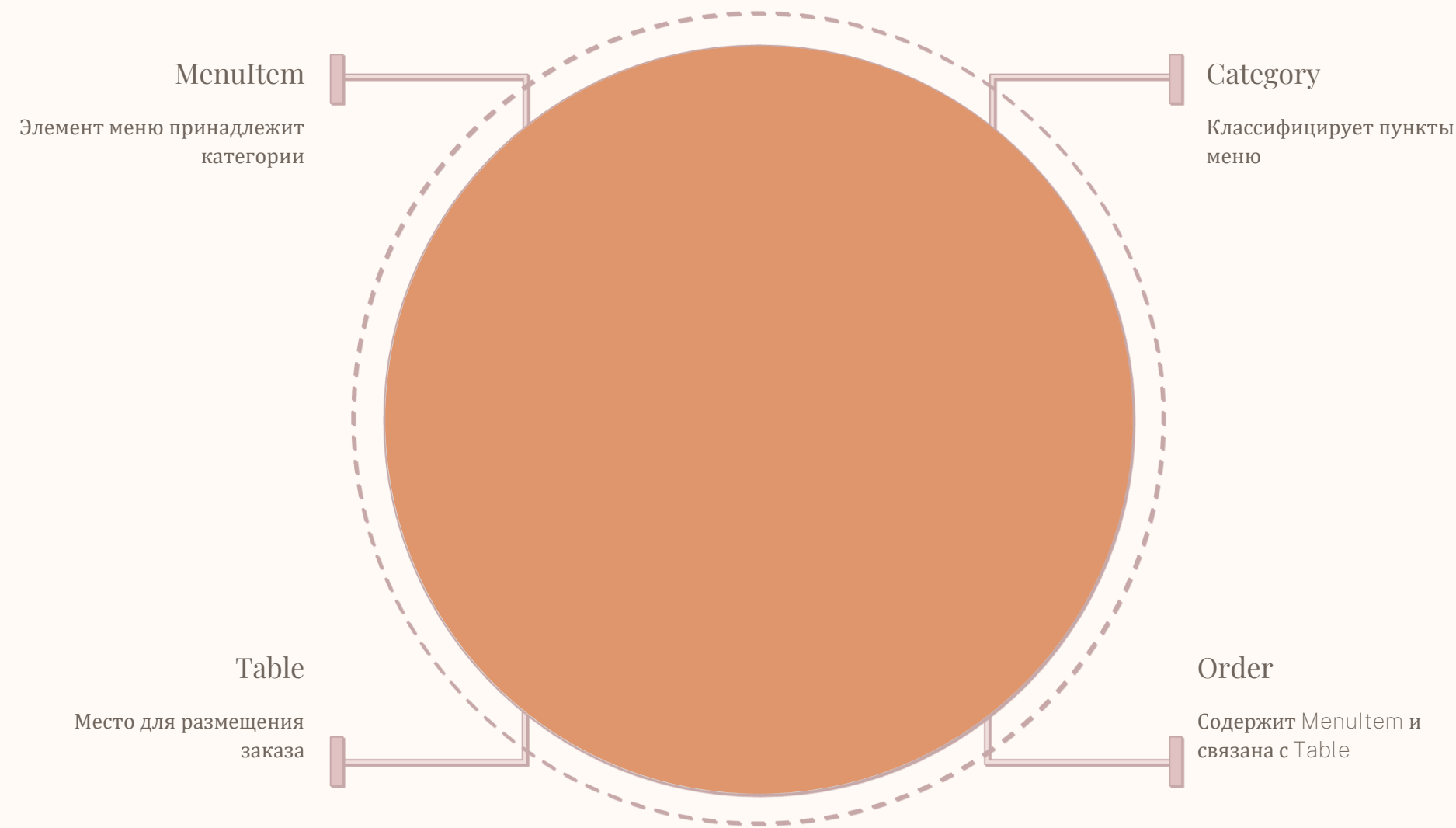
## Payment

id, orderId, amount, method, status.



# Взаимодействие между сущностями

Модель данных разработана для обеспечения эффективного взаимодействия между различными элементами системы учета заказов.



Каждая сущность играет ключевую роль в общей функциональности системы, обеспечивая целостность и согласованность данных.

# Запуск проекта

Для локального запуска проекта необходимо выполнить несколько простых шагов.

01

## Клонирование репозитория

Загрузите исходный код проекта из репозитория.

02

## Установка зависимостей

```
npm install
```

Установите все необходимые пакеты и библиотеки.

03

## Настройка Firebase

Сконфигурируйте Firebase Realtime Database в файле `firebase.ts`.

04

## Запуск Development сервера

```
npm run dev
```

Запустите сервер для разработки и начните работу с приложением.

